

COMPUTER ARCHITECTURE



LECTURE 12: Associative cache

CONTENTS

- > Cache performance
 - Impacts of cache misses
 - Average Memory Access Time (AMAT)
- > Associative cache
 - Cache associativity concept
 - Replacement policies
- > Multi-level caches
 - Performance analysis

Impacts Cache Misses on Performance

- ❑ The processor stalls on a cache miss
 - When fetching instructions from the **Instruction Cache (I-cache)**
 - When loading or storing data into the **Data Cache (D-cache)**
 - Miss penalty is assumed equal for I-cache & D-cache
 - Miss penalty is assumed equal for Load and Store
- ❑ Components of CPU time:
 - Program execution cycles (includes cache hit time)
 - Memory stall cycles (mainly from cache misses)
 - $\text{CPU time} = \text{IC} \times \text{CPI} \times \text{CC} = \text{IC} \times (\text{CPI}_{\text{ideal}} + \text{Memory-stall cycles}) \times \text{CC}$
 - $\text{CPI}_{\text{ideal}}$ = CPI for ideal cache (no cache misses)
 - $\text{CPI}_{\text{stall}}$ = CPI in the presence of memory stalls
 - Memory stall cycles increase the CPI!

Memory Stall Cycles

❑ Stall cycles = (I-Cache Misses + D-Cache Misses) × Miss Penalty

➤ I-Cache Misses = I-Count × I-Cache Miss Rate

➤ D-Cache Misses = LS-Count × D-Cache Miss Rate

▪ LS-Count (Load & Store) = I-Count × LS Frequency

❑ With simplifying assumptions:

Memory stall cycles = I-Count × misses/instruction × miss penalty

➤ Memory stall cycles/instruction = I-Cache Miss Rate × Miss Penalty +
LS Frequency × D-Cache Miss Rate × Miss Penalty

➤ For write-through caches: $\underbrace{\text{I-Cache Miss Rate} + \text{LS Frequency} \times \text{D-Cache Miss Rate}}_{\text{miss rate}} \times \text{miss penalty}$

Memory Stall Cycles: Example

❑ Example: Compute misses/instruction and memory stall cycles for a program with the given characteristics

- Instruction count (I-Count) = 10^6 instructions
- 30% of instructions are loads and stores
- D-cache miss rate is 5% and I-cache miss rate is 1%
- Miss penalty is 100 clock cycles for instruction and data caches

❑ Solution:

- misses/instruction = $1\% + 30\% \times 5\% = 0.025$;
- memory stall cycles/instruction = $0.025 \times 100 = 2.5$ cycles
- total memory stall cycles = $2.5 \times 10^6 = 2,500,000$ cycles

Relative Impacts of Stalls

- ❑ Relative cache penalty **increases** as processor performance **improves** (faster clock rate and/or lower CPI)
 - When calculating CPI_{stall} , the cache miss penalty is measured in processor clock cycles needed to handle a miss.
 - The lower the CPI_{ideal} , the more pronounced the impact of stalls

- ❑ Example: Given

- I-cache miss rate = 2%, D-cache miss rate = 4%
 - Miss penalty = 100 cycles
 - Base CPI (ideal cache) = 2
 - Load & stores are 36% of instructions

Questions:

- What is CPI_{stall} ? $2 + (2\% + 36\% \times 4\%) \times 100 = 5.44$, % time on memory stall = 63%
- What if the CPI_{ideal} is reduced to 1? % time on memory stall = 77%
- What if the processor clock rate is doubled? Miss penalty = 200, $CPI_{stall} = 8.88$

Average Memory Access Time (AMAT)

❑ Hit time is also important for performance

- A larger cache will have a longer access time → an increase in hit time will likely add another stage to the pipeline.
- At some point, the increase in hit time for a larger cache will overcome the improvement in hit rate leading to a decrease in performance.

❑ Average Memory Access Time (AMAT) is the average time to access memory considering both hits and misses.

$$\text{AMAT} = \text{Hit time} + \text{Miss rate} \times \text{Miss penalty}$$

❑ Example: Find the AMAT for a cache with

- Cache access time (Hit time) of 1 cycle = 2 ns
- Miss penalty of 20 clock cycles
- Miss rate of 0.05 per access

❑ Solution:

- $\text{AMAT} = 1 + 0.05 \times 20 = 2 \text{ cycles} = 4 \text{ ns}$
- Without the cache, AMAT will be equal to miss penalty = 20 cycles = 40 ns

Cache Associativity Concept

❑ More flexible block placement?

- In a **direct mapped** cache a memory block maps to **exactly one** cache block
- At the other extreme, could allow a memory block to be mapped to **any** cache block → **fully associative cache** (no indexing)

❑ A compromise is to divide the cache into **sets**, each of which consists of n “ways” (**n-way set associative**).

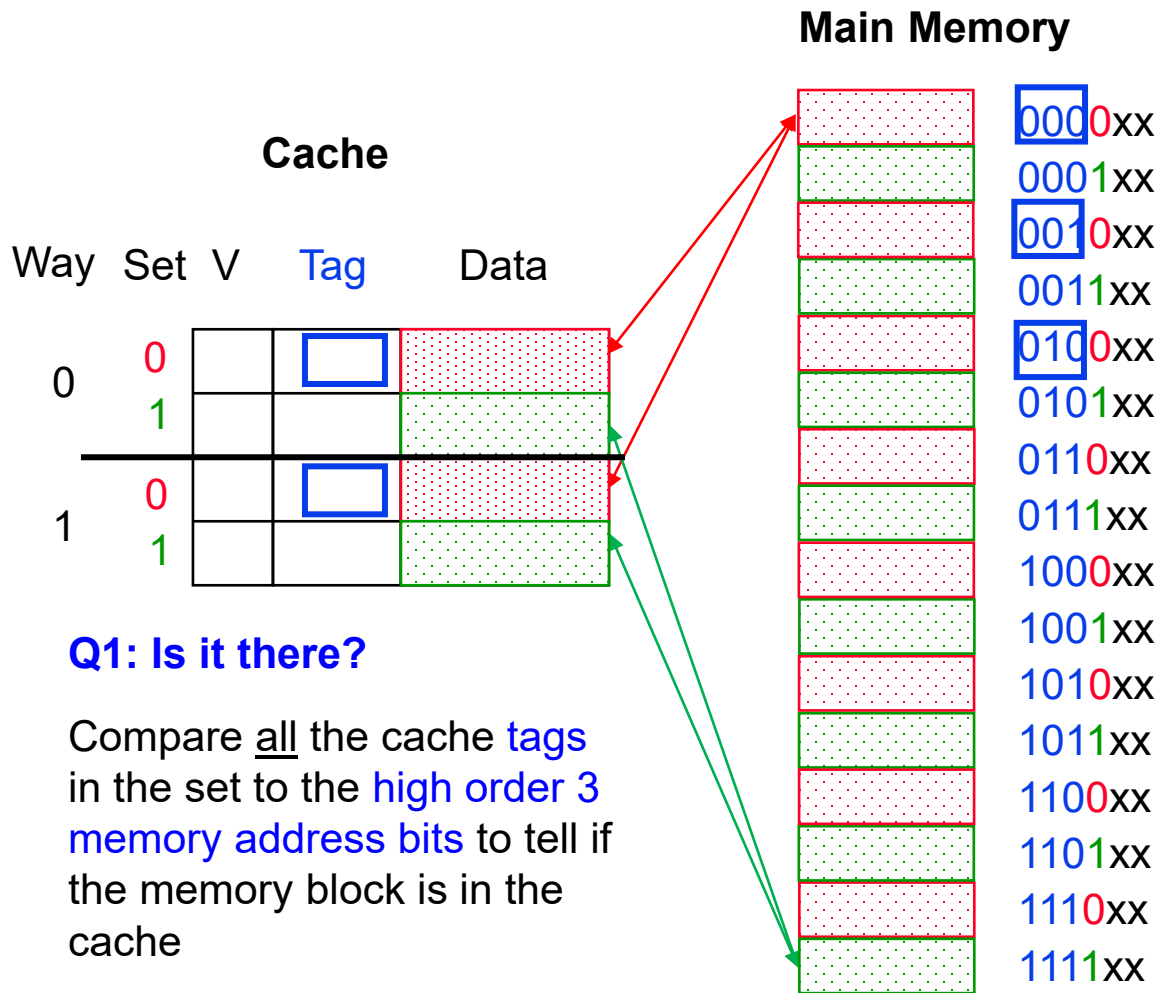
- A memory block maps to a unique set (specified by the **index** field) and can be placed in any way of that set (so there are n choices).

$$\text{Set index} = (\text{block address}) \bmod (\# \text{ sets in the cache})$$

❑ Example: design a 1-word block, 2-way set associative cache

- Q1: Is the data there?
- Q2: If the data is in the cache, how do we find it?

Two-way Set Associative Cache



One-word blocks
Two low order bits define the byte in the word (32b words)

Q2: How do we find it?

Use **next 1 low order memory address bit** to determine which cache set (i.e., modulo the number of sets in the cache)

Two-way Set Associative Cache: Miss Rate

- Consider the main memory word reference for the following string

0 4 0 4 0 4 0 4

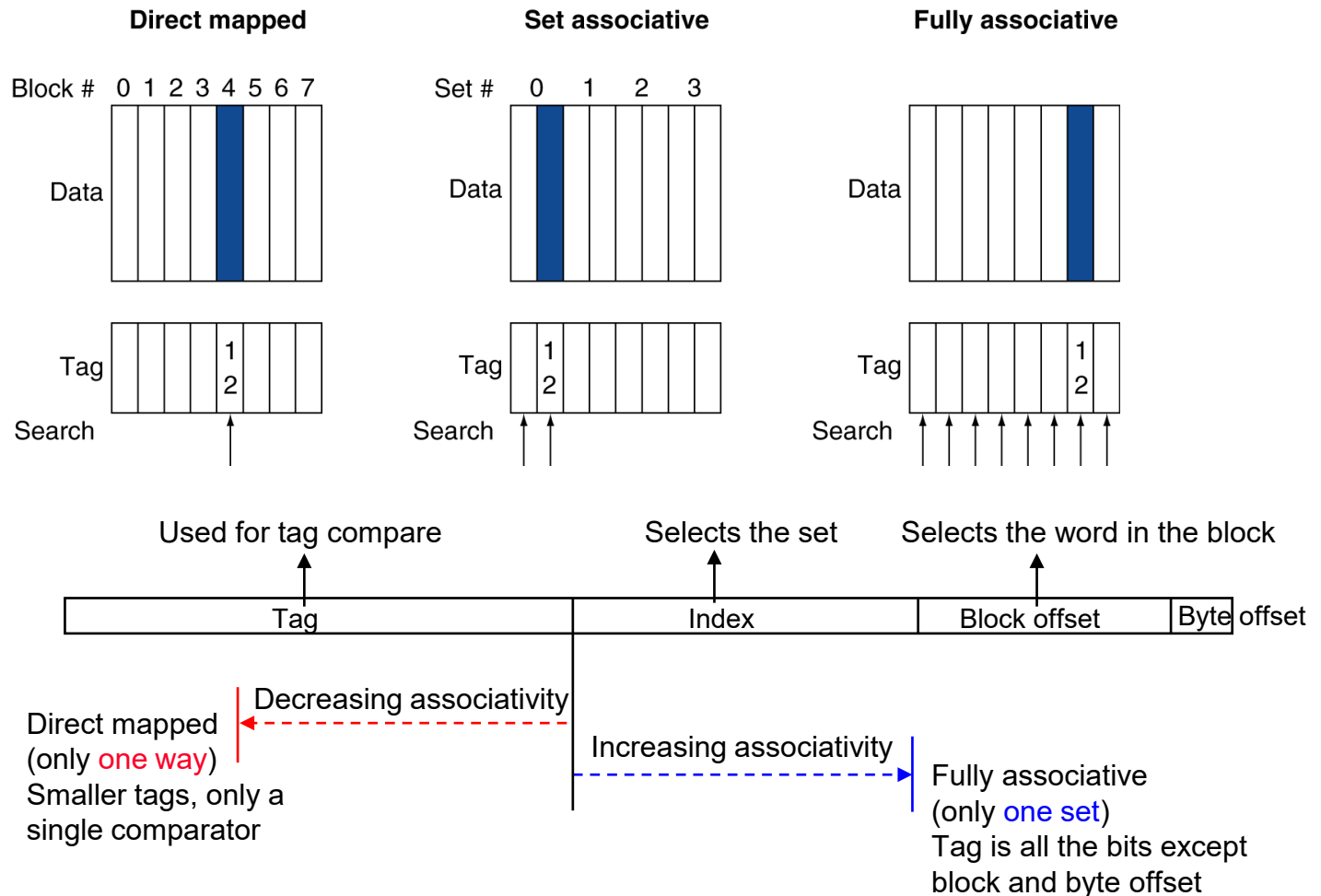
- Same capacity as in direct-mapped example, but increase associativity to 2
- Start with an empty cache - all blocks initially marked as not valid

0 miss	4 miss	0 hit	4 hit																																
<table><tr><td>000</td><td>Mem(0)</td></tr><tr><td></td><td></td></tr><tr><td></td><td></td></tr><tr><td></td><td></td></tr></table>	000	Mem(0)							<table><tr><td>000</td><td>Mem(0)</td></tr><tr><td></td><td></td></tr><tr><td>010</td><td>Mem(4)</td></tr><tr><td></td><td></td></tr></table>	000	Mem(0)			010	Mem(4)			<table><tr><td>000</td><td>Mem(0)</td></tr><tr><td></td><td></td></tr><tr><td>010</td><td>Mem(4)</td></tr><tr><td></td><td></td></tr></table>	000	Mem(0)			010	Mem(4)			<table><tr><td>000</td><td>Mem(0)</td></tr><tr><td></td><td></td></tr><tr><td>010</td><td>Mem(4)</td></tr><tr><td></td><td></td></tr></table>	000	Mem(0)			010	Mem(4)		
000	Mem(0)																																		
000	Mem(0)																																		
010	Mem(4)																																		
000	Mem(0)																																		
010	Mem(4)																																		
000	Mem(0)																																		
010	Mem(4)																																		

- 8 requests, 2 misses

- Solves the ping pong effect in a direct mapped cache since now 2 memory locations that map into the same cache set can co-exist!

Spectrum of Associativity



Replacement Policies

- ❑ A miss occurred, which way's block do we pick for replacement?
 - Direct mapped: no choice.
 - Set associative: non-valid entry, then **choose among entries** in the set.
- ❑ First In First Out (**FIFO**): replace the oldest block in set
 - Use one counter per set to specify the oldest block. On a cache miss replace the block specified by counter & increment the counter.
- ❑ Least Recently Used (**LRU**): replace the longest unused
 - Track when each way's block was used relative to the other in the set by HW
 - E.g. use one bit per set for a 2-way associative cache → set the bit when a block is referenced and reset the other way's bit.
 - Manageable for 4-way, too hard beyond that.
- ❑ Random
 - Gives approximately the same performance as LRU for high associativity.

Example: Matrix Multiplication (1)

❑ Matrix multiplication: $C = A * B$

- Each is a 4x4 matrix, elements are 4 bytes
- B has been transposed into Bt to optimize efficiency
- Matrix A stored at 0x1000
- Matrix B stored at 0x2000
- Matrix C stored at 0x3000
- No other memory used

Bt

4	4	4	4
3	3	3	3
2	2	2	2
1	1	1	1

A

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

C

❑ Cache is **full associative**

- Block size is 16 bytes (a row)
- **Write allocate**
- **Write back with LRU**
- Lower LRU = more recent used
- Cache starts cold: Random data, but all valid bits are zero.

Tag	V	Dirty	LRU	Data
0x100	0	0	0	0xBF 0x16 0x88 0x2B
0x733	0	0	0	0x3B 0x18 0xF1 0xB3
0x156	0	0	0	0xE6 0x57 0x49 0xEE
0x4E9	0	0	0	0xB5 0x81 0x67 0x3F

Example: Matrix Multiplication (2)

❑ Compute C[0]

- Read A[0] = 0x1000: miss → copy block A[0:3] to cache, set Tag, V, LRU bits
- Read Bt[0] = 0x2000: miss → copy block B[0:3] to cache, set Tag, V, LRU bits
- Read A[1] = 0x1004: hit → update LRU bits
- ...
- Read Bt[3] = 0x200C: hit → update LRU bits

❑ Write to C[0] = 0x3000: Miss, write allocate

- Write back, set dirty bit and update Tag, V, LRU bits
- Main memory **not** updated yet

Tag	V	Dirty	LRU	Data
0x100	0	0	0	0xBF 0x16 0x88 0x2B
0x733	0	0	0	0x3B 0x18 0xF1 0xB3
0x156	0	0	0	0xE6 0x57 0x49 0xEE
0x4E9	0	0	0	0xB5 0x81 0x67 0x3F

A

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

Bt

4	4	4	4
3	3	3	3
2	2	2	2
1	1	1	1

C

Example: Matrix Multiplication (3)

❑ Compute C[1]

- Access A[0] = 0x1000: hit → update LRU bits, Access Bt[4] = 0x2010: miss
 - Copy block Bt[4:7] to cache, update necessary bits
 - Access A[1] = 0x1004: hit, Access Bt[5] = 0x2014: hit
 - Access A[2] = 0x1008: hit, Access Bt[6] = 0x2018: hit
 - Access A[3] = 0x100C: hit, Access Bt[7] = 0x201C: hit
- } Update LRU bits

❑ Write to C[1] = 0x3004: Hit

- Write back, update LRU bits
- Main memory **not** updated yet

Tag	V	Dirty	LRU	Data
0x100	1	0	1	0x01 0x02 0x03 0x04
0x200	1	0	3	0x04 0x04 0x04 0x04
0x300	1	1	2	0x28 xxxx xxxx xxxx
0x4E9	0	0	0	0xB5 0x81 0x67 0x3F

A

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

Bt

4	4	4	4
3	3	3	3
2	2	2	2
1	1	1	1

C

Example: Matrix Multiplication (4)

❑ Compute C[2]

- Access A[0] = 0x1000: hit, Access Bt[8] = 0x2020: miss
 - Cache full → evict block 0x200 (which has highest LRU)
 - copy block Bt[8:11] to cache, update necessary bits
 - Access A[1] = 0x1004: hit, Access Bt[9] = 0x2024: hit
 - Access A[2] = 0x1008: hit, Access Bt[10] = 0x2028: hit
 - Access A[3] = 0x100C: hit, Access Bt[11] = 0x202C: hit
- } Update LRU bits

❑ Write to C[2] = 0x3008: Hit

- Write back, update LRU, main memory **not** updated yet.

Tag	V	Dirty	LRU	Data
0x100	1	0	1	0x01 0x02 0x03 0x04
0x200	1	0	4	0x04 0x04 0x04 0x04
0x300	1	1	2	0x28 0x1E xxxx xxxx
0x201	1	0	3	0x03 0x03 0x03 0x03

A

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

Bt

4	4	4	4
3	3	3	3
2	2	2	2
1	1	1	1

C

Example: Matrix Multiplication (5)

❑ Compute C[3]

- Access A[0] = 0x1000: hit, Access Bt[12] = 0x2030:
 - miss, cache full → evict block 0x201
 - copy block Bt[8:11] to cache, update necessary bits
- Access A[1] = 0x1004: hit, Access Bt[13] = 0x2024: hit
- Access A[2] = 0x1008: hit, Access Bt[14] = 0x2028: hit
- Access A[3] = 0x100C: hit, Access Bt[15] = 0x202C: hit

} Update LRU bits

❑ Write to C[3] = 0x300C:

- Hit, update LRU bits, main memory **not** updated yet.

Tag	V	Dirty	LRU	Data
0x100	1	0	3	0x01 0x02 0x03 0x04
0x202	1	0	4	0x02 0x02 0x02 0x02
0x300	1	1	1	0x28 0x1E 0x14 0x0A
0x203	1	0	2	0x01 0x01 0x01 0x01

A

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

Bt

4	4	4	4
3	3	3	3
2	2	2	2
1	1	1	1

C

Example: Matrix Multiplication (6)

❑ Compute C[4]

- Access A[4] = 0x1010: miss
 - Cache full → evict block 0x202
 - copy block A[4:7] to cache, update necessary bits
- Access Bt[0] = 0x2000: miss
 - Cache full → evict block 0x100
 - copy block Bt[0:3] to cache, update necessary bits
- Other accesses: hits, update LRU bits
- Write to C[4] = 0x3010: miss → evict block 0x203
 - Write back, memory **not** updated yet.

Tag	V	Dirty	LRU	Data
0x100	1	0	3	0x01 0x02 0x03 0x04
0x202	1	0	4	0x02 0x02 0x02 0x02
0x300	1	1	1	0x28 0x1E 0x14 0x0A
0x203	1	0	2	0x01 0x01 0x01 0x01

A

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

Bt

4	4	4	4
3	3	3	3
2	2	2	2
1	1	1	1

C

Example: Matrix Multiplication (7)

□ Compute C[5]

- Access A[4] = 0x1010: hit, Access Bt[4] = 0x2010: miss
 - Evict block 0x300, dirty data: **main memory get updated.**
 - Copy block Bt[4:7] to cache, update necessary bits
- Access A[5] = 0x1014: hit, Access Bt[5] = 0x2014: hit
- Access A[6] = 0x1018: hit, Access Bt[6] = 0x2018: hit
- Access A[7] = 0x101C: hit, Access Bt[7] = 0x201C: hit
- Write to C[5] = 0x3014: hit
 - Write back, update LRU, main memory **not** updated yet.

Bt

4	4	4	4
3	3	3	3
2	2	2	2
1	1	1	1

Tag	V	Dirty	LRU	Data
0x200	1	0	3	0x04 0x04 0x04 0x04
0x101	1	0	1	0x05 0x06 0x07 0x08
0x300	1	1	4	0x28 0x1E 0x14 0x0A
0x301	1	1	2	0x68 xxxx xxxx xxxx

A

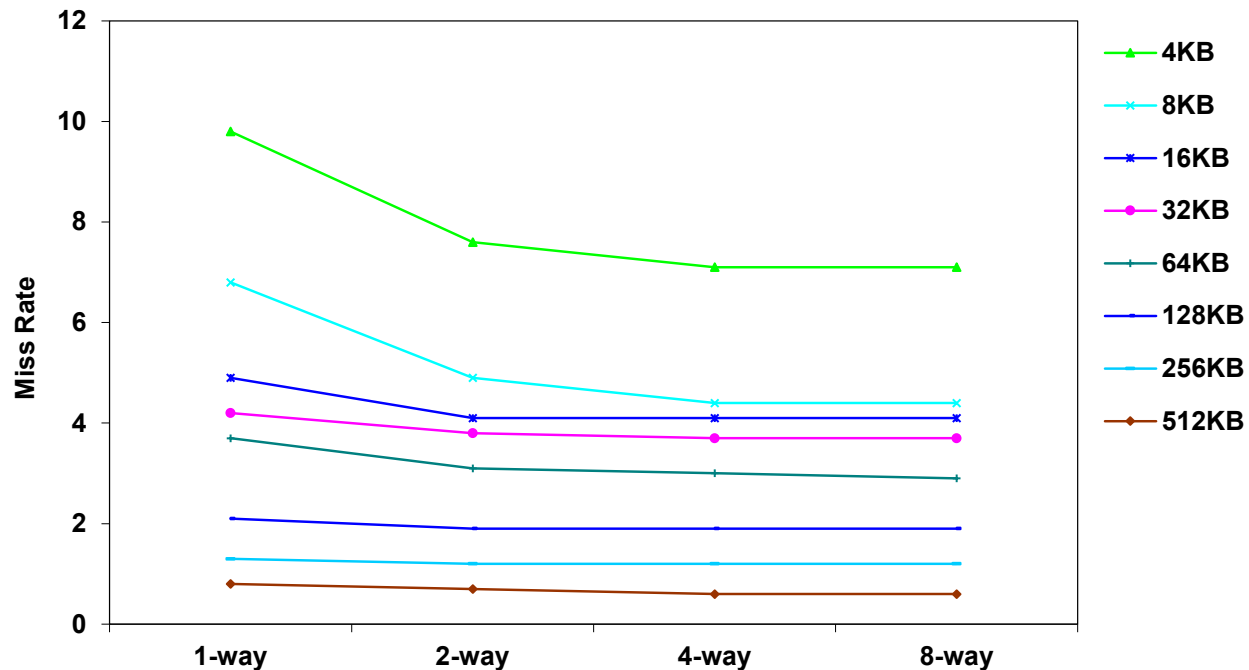
1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

C

40	30	20	10

How Much Associativity?

- ❑ Increased associativity decreases miss rate
 - But with diminishing returns



- ❑ Balancing cost of a miss versus the cost of implementation
 - Area (for comparators + MUX), delay (due to comparators + MUX)

Multi-level Caches

- ❑ Using multiple levels of caches can reduce miss rate
 - Primary (L1) cache attached to CPU
 - Larger, slower, L2 cache services misses from primary cache.
 - in some cases, even a unified L3 cache on the die.
- ❑ **Example:** given
 - CPU base CPI = 1, clock rate = 4GHz
 - Miss rate/instruction = 2%
 - Main memory access time = 100ns

Questions:

- Compute the actual CPI with just primary cache.
- Compute the performance gain if we add L2 cache with
 - Access time = 5ns
 - Global miss rate to main memory = 0.5%

Multi-level Caches (2)

❑ With just primary cache

➤ Miss penalty = $100\text{ns}/0.25\text{ns} = 400$ cycles

➤ $\text{CPI}_{\text{stall}} = 1 + 0.02 \times 400 = 9$

❑ With added L2 cache

➤ Primary miss, L2 hit: penalty = $5\text{ns}/0.25\text{ns} = 20$ cycles

➤ Primary miss, L2 miss: penalty = L2 access stall + Main memory stall = $20 + 400 = 420$ cycles

➤ $\text{CPI}_{\text{stall}} = 1 + (\text{Stalls due to L1 miss, L2 hit}) + (\text{Stalls due to L1 miss, L2 miss}) = 1 + 0.02 \times 0.995 \times 20 + 0.02 \times 0.005 \times 420 = 1.44$ cycles.

➤ Alternatively, $\text{CPI}_{\text{stall}} = \text{L1 stalls/instruction} + \text{L2 stalls/instruction} = 1 + 0.02 \times 20 + 0.02 \times 0.005 \times 400 = 1.44$ cycles.

➤ Performance gain = $9/1.44 = 6.25$ times.

Multilevel Cache Design Considerations

- ❑ Design considerations for L1 and L2 caches are very different
 - Primary cache should focus on **minimizing hit time** in support of a shorter clock cycle → smaller with smaller block sizes.
 - Secondary cache(s) should focus on **reducing miss rate** to reduce the penalty of long main memory access times → larger with larger block sizes & higher levels of associativity.
- ❑ The miss penalty of the L1 cache is significantly reduced by the presence of an L2 cache – so it can be smaller (i.e., faster) but have a higher miss rate
- ❑ For the L2 cache, hit time is less important than miss rate
 - The L2\$ hit time determines L1\$'s miss penalty
 - L2\$ **local miss rate** >> the **global miss rate**
 - **Local miss rate** = fraction of references to one level of a cache that miss
 - **Global miss rate** = fraction of references that miss in all levels of a multi-level cache → dictates how often we must access the main memory.

Multi-level Caches: Real-life Examples

	Intel Nehalem	AMD Barcelona
L1 cache organization & size	Split I\$ and D\$; 32KB for each per core; 64B blocks	Split I\$ and D\$; 64KB for each per core; 64B blocks
L1 associativity	4-way (I), 8-way (D) set assoc.; ~LRU replacement	2-way set assoc.; LRU replacement
L1 write policy	write-back, write-allocate	write-back, write-allocate
L2 cache organization & size	Unified; 256MB (0.25MB) per core; 64B blocks	Unified; 512KB (0.5MB) per core; 64B blocks
L2 associativity	8-way set assoc.; ~LRU	16-way set assoc.; ~LRU
L2 write policy	write-back, write-allocate	write-back, write-allocate
L3 cache organization & size	Unified; 8192KB (8MB) shared by cores; 64B blocks	Unified; 2048KB (2MB) shared by cores; 64B blocks
L3 associativity	16-way set assoc.	32-way set assoc.; evict block shared by fewest cores
L3 write policy	write-back, write-allocate	write-back; write-allocate

The Cache Design Space

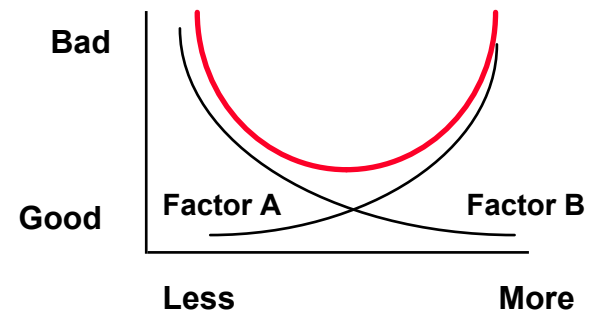
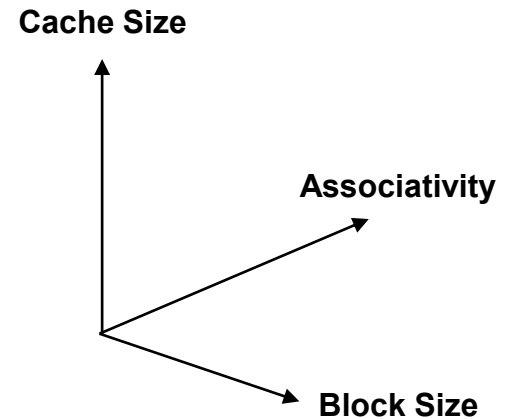
❑ Several interacting dimensions

- cache size
- block size
- associativity
- replacement policy
- write-through vs write-back
- write allocation

❑ The optimal choice is a compromise

- depends on access characteristics
 - workload
 - use (I-cache, D-cache, TLB)
- depends on technology / cost

❑ Simplicity often wins



CONCLUSION

Key takeaways

- > Performance impacted by cache
 - AMAT reflects the total impacts by both hits & misses
- > Associative cache
 - A memory block maps to any way of a unique set
 - Helps reduces cache misses
- > Multi-level caches
 - Help reduces cache misses
 - Design considerations are different at each level

NEXT LECTURE

[Flipped class] Virtual memory

- > Pre-class
 - Homework
 - Self study
- > In class
 - Reinforcement/enrichment discussion
 - Quizzes
- > Post class
 - Homework
 - Consultation (if needed)